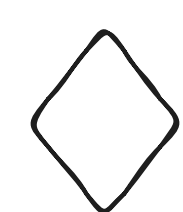
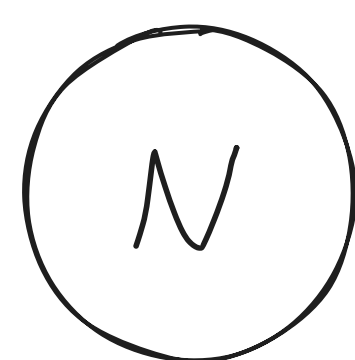


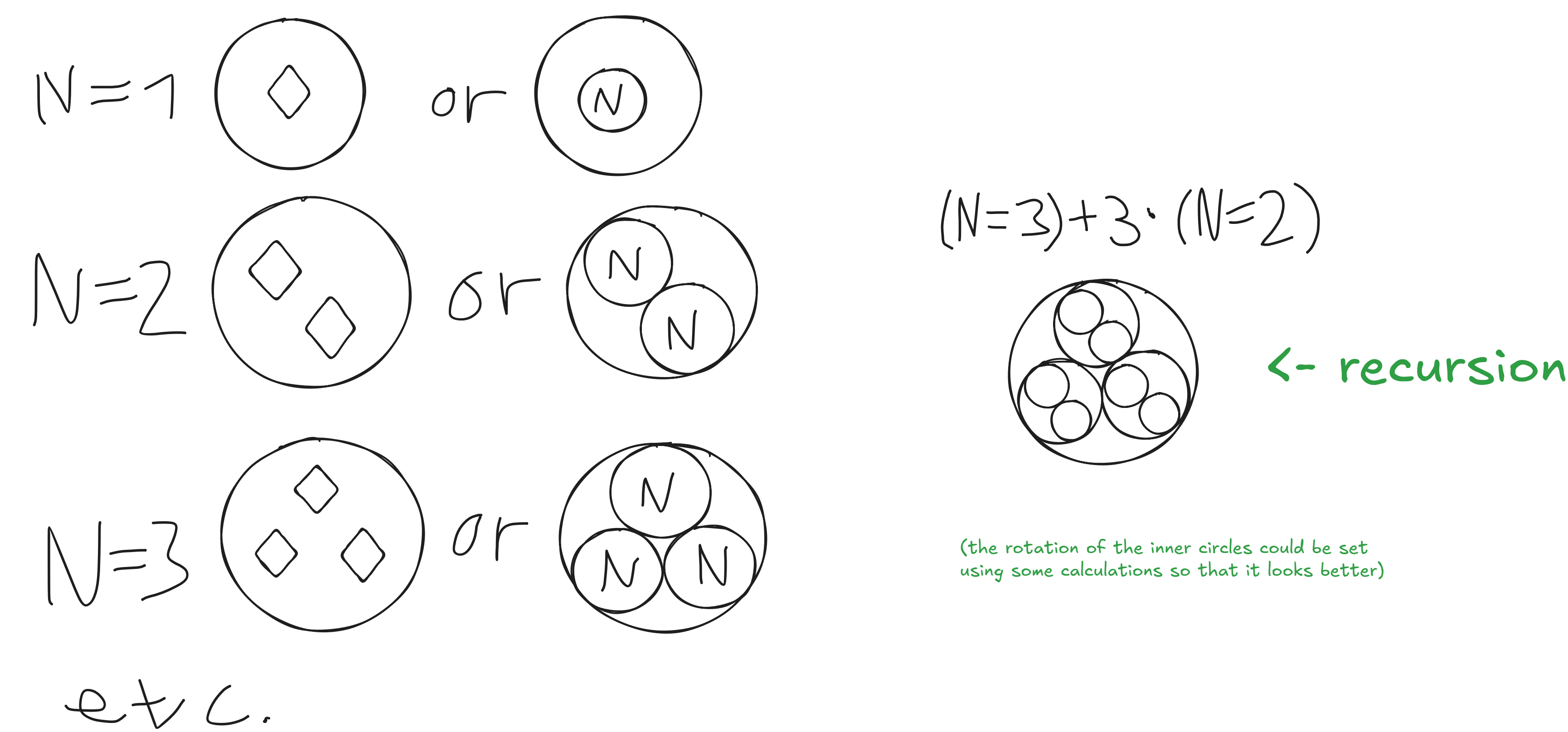
My ideas for a magic system datapack

Theory

Ingredient = 

Circle = 

Each parent circle has N children.
A child can be another circle or an ingredient.
An ingredient can be anything,
but it will probably be something displayed at this position
and the outer circle will use it for its magic effect.



Thinking of parents and children will make it easier
to talk about each element in an abstract way.
A parent has control/knowledge of their children, but not over anything else.

A fire ball spell might be a circle with
a ball ingredient, an element ingredient and a direction ingredient.
The parent could be a summon function that requires
one ingredient to be summonable and any other ingredients as effects.
The children would be the 3 ingredients.

In the creation phase,
an mcfuction summons the circles and ingredient displays.
In the execution phase, the summon function summons the ball,
and the ball will have the effects of fire particles and movement
in the effects phase (using entity tags).

With this logic,
there is no limit to how many combinations we can make.
We will use recursion to program the creation and execution phase.

Recursion simply means that a thing can be inside itself.
There could be infinitely many circles inside each other, but the computer needs a stop condition.
Minecraft raycasting is also done with recursion, but each step (position) will only create one new step (new position).
Here, one step (circle) can have multiple sub-steps (circles).

If we want to store a magic spell as an item,
the item's custom data will look like this:

```
{
  size: 5
  function: go up / summon / ...
  children: [
    { size: ..., function: ..., children: [...] }, <- recursion
    { ... },
    ...
  ]
}
```

This data structure is known as a parent-child tree.
Since the child and the parent can both be circles, this structure is recursive.
Realistically, only the innermost spells can have ingredients.
But it could also be programmed differently depending on the magic system.

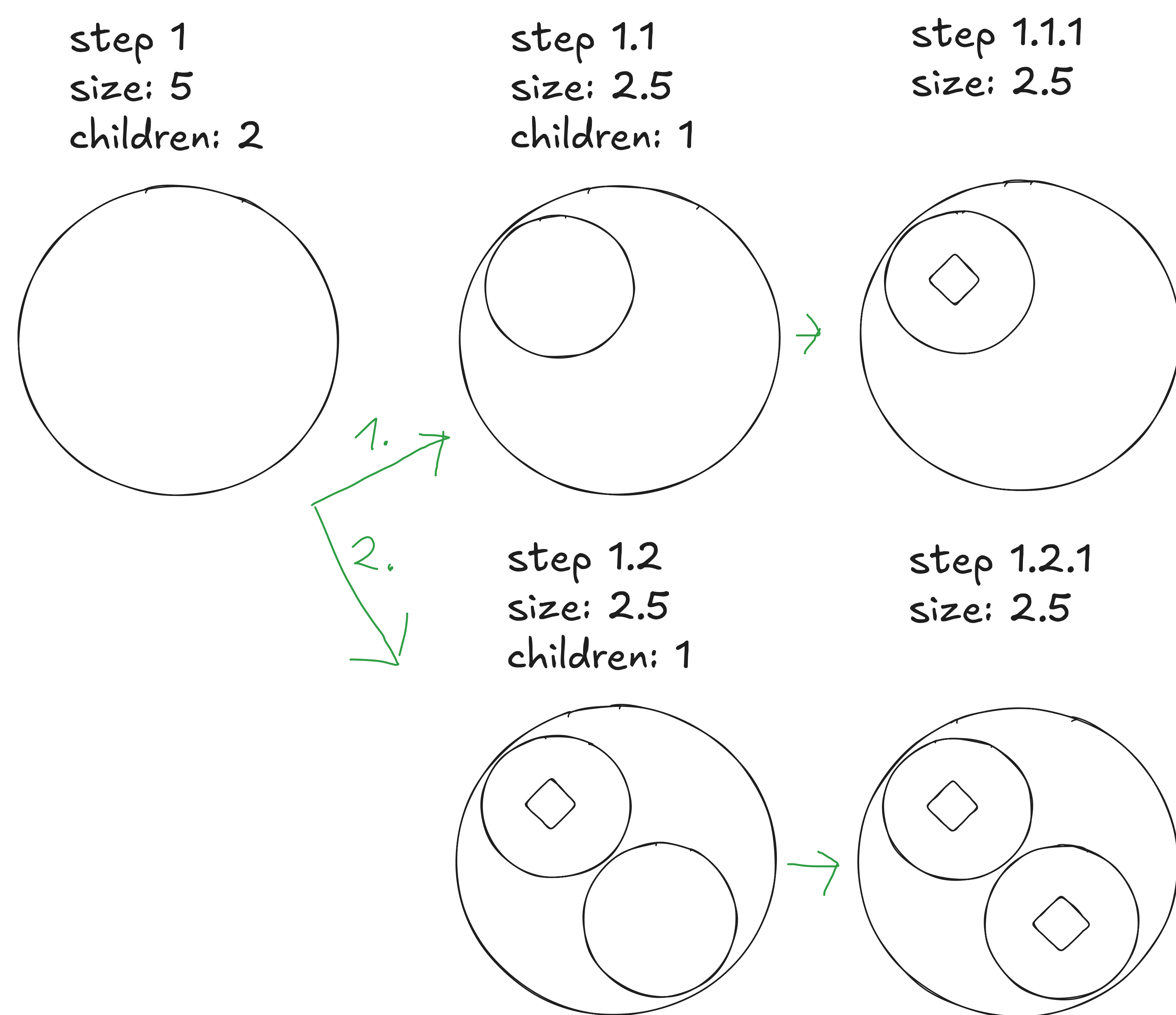
Creation phase

summon the circle at the specific size at the current position
for each child, at an offset position
if its a circle, do the whole creation phase for it <- recursion
if its an ingredient, summon

each child will

- be x times smaller eg. 3 children -> 46.4% size of parent
- execute at an offset position
- receive tags based on the parent function
eg. go up, have particles, do damage...
children's children may inherit tags like go up
so that the whole magic goes up with one mcfuction file

Example: 1 circle containing 2 circles containing 1 ingredient



this would be equal to the following custom data:

```
{  
  size: 5,  
  children: [  
    { children: [ ingredient_1 ] },  
    { children: [ ingredient_2 ] }  
  ]  
}
```

to have simpler code (not necessary):
a macro for size and position may be used
a macro to iterate over the list of children may be used (the code is shown later)

Execution phase

starting with the outmost circle
if its a circle with ingredients,
create a circle entity using ingredients (like a ball or just something that keeps track of range)
if its a circle with circles:
for each child
do the execution phase <- recursion
then do something to the inner entities (like giving them the same effect tag)

Effect phase

tick.mcfunction: with this pattern, @e is only searched once
as @e[tag = magic_effect, ...] run magic_effect.mcfunction

magic_effect.mcfunction:
 as @s[tag = go up] run function go up
 as @s[tag = particles] run function particles
 etc.

go up.mcfunction
 tp @s ~ ~0.1 ~ OR data modify @s Motion[1] OR similar

How can spells that use multiple circles be handled?

Imagine that you are the parent of 3 fire balls
and want them to rotate around the same center.
During the execution phase, you must give them an id/score,
so that you can still know where they are
when you are running rotate.mcfunction during the effect phase.
(like a birth certificate or family name ^^)

IF a global score is used for this, it is advised to increment it for every new spell and simply reset the global max id on load if no spell exists in that moment

How to recursively iterate a list in minecraft (needed for the pseudocode "for each child"):
The iteration method used is called
"preorder tree traversal using a stack"

start.mcfuction:

```
# add the first child to the stack to start with it
data modify storage data stack set value []
data modify storage data stack prepend from # some item/entity
function recursion
```

recursion.mcfuction:

```
# pop the next child from the stack
data modify storage data current set from storage data stack[0]
data remove storage data stack[0]

# do stuff with the current child

# add all the children to the stack
execute if data storage data current.children[0] run function append_children

# continue if there are still children on the stack
execute if data storage data stack[0] run function recursion
```

append_children.mcfuction

```
data modify storage data stack append from storage data current.children[0]
data remove storage data current.children[0]
# (execute store result storage data stack[0].xyz... # some data that i want my children to inherit)
execute if data storage data current.children[0] run function append_children
```

How to iterate a normal list:

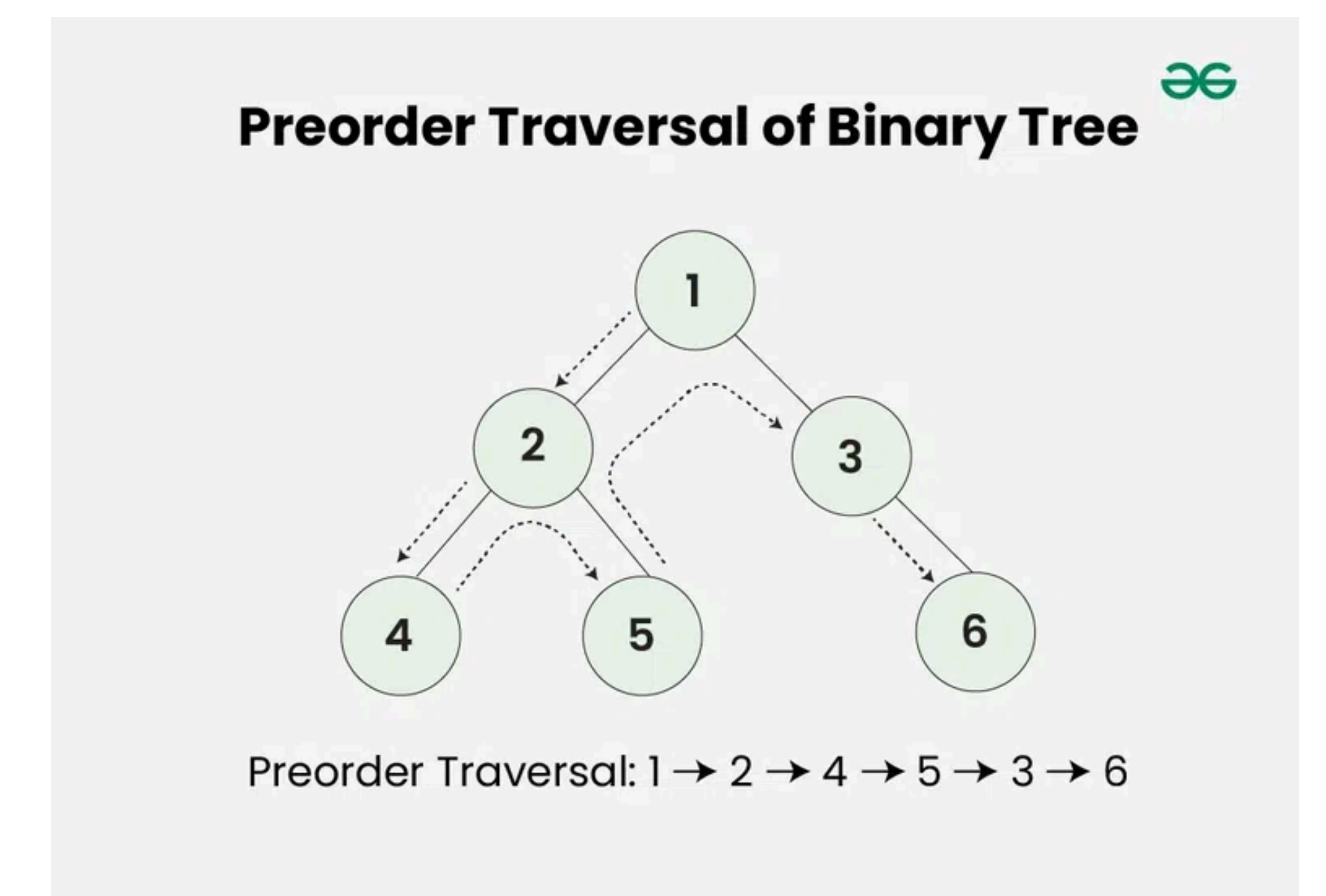
start.mcfuction:

```
data modify storage data temp set from # some source
execute if data storage data temp[0] run function iterate_list
```

iterate_list.mcfuction:

```
# do something with temp[0]
```

```
data remove storage data temp[0]
execute if data storage data temp[0] run function iterate_list
```



Generation/Crafting of spells

If you already had some sub-spells,
you would create a new outer spell and add the sub spells as children.
The player will never create multiple layers in one step, and only add layer by layer.
This makes crafting much easier than the rest of the code.

Lets say we have the old spell data in one place (item or entity),
and the new one in another place. The code would look like this:

```
data modify new_item.custom_data.children append from old_item_1.custom_data
```

Duration of phases

The creation and execution phase can be triggered by the player.
The effect phase happens as soon as the execution phase is done.
How does the effect phase end? Here are some suggestions:

- Default time
- Cancelling by the player (like not holding a magic staff up anymore)
- Time based on power added. The outermost circle has a property that stores how long until death.
- Time based on power added. Each circle entity can keep track of its time. (Like the fireball and the waterball could have different animations that last different amounts of time. Or you could have 20 magic bats that don't die at the same time)

Depending on the method, all entities that belong to a spell OR all entities of the same family would have to be tagged so that they can be killed at a certain time.

Since some spells want to know where their children are anyways,
I suggest to group all children with global ids during the summoning phase tree traversal code that I wrote. Each element will increment to the next score and give itself and its immediate children this score.
(see "some data that I want my children to inherit")

Spell Ideas

- Elements
- Directions
- Charge
- Longevity
- Shape (Ball, Line, Area, Many small balls, Cone)
- Explosion when hitting something
- Potion Effect when hitting someone
- Solid Objects (Wall, Block)
- Particle Effects (Smoke, Light, Party)
- Disappear when hitting someone or bounce on walls
- Destruction of blocks
- Placement of block(s) (like flowers or water)
- Custom potion effect that don't exist in minecraft like transformation

Animations, Timing

We probably want to keep the spell circle visible for a while after executing it. This means that we need to have a disabled state where all display entities are tagged as such.

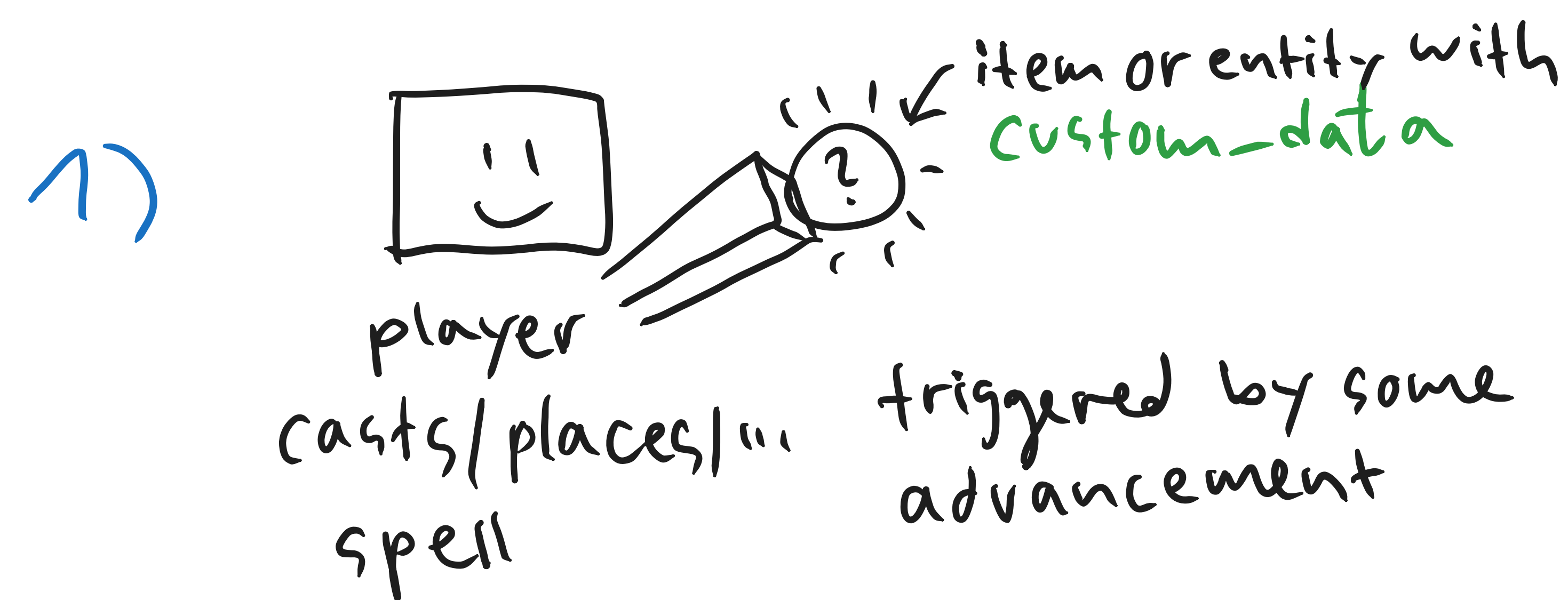
All display entities also need a global spell id so that the whole magic circle can be deleted at once.

The same goes for the effect entities, and as said before, they should also have an id for their family.

Since the execution phase might take a while to do animations, there has to be a tag that marks that state too. And this state has to be taken care of in tick.mcfunction just like effects are.

What are ways to keep track of animation timing? The easiest is a scoreboard, but it is not stackable on a single entity. A good solution could be a tag on the entity and a timer that is the same for the whole spell.

Detailed View of all Phases focussing on Entities

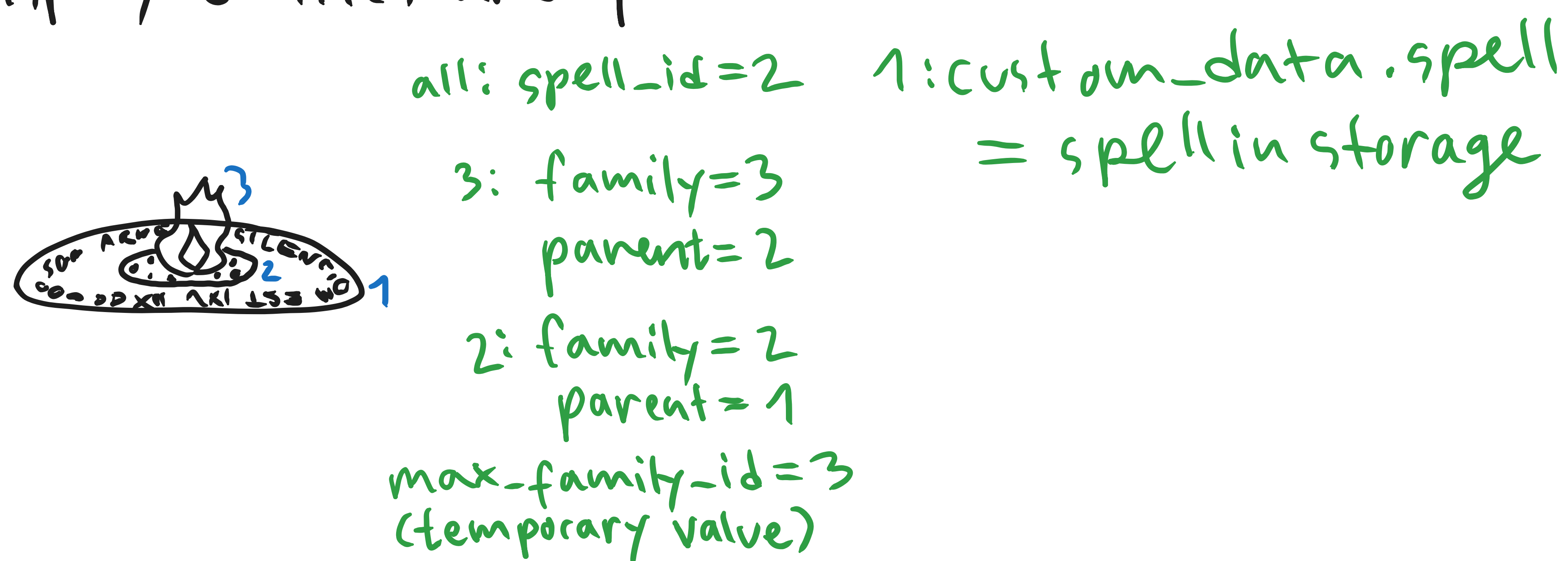


2) the `custom_data` is loaded into a storage `data.spell`

3) `create.mfunction` is called

4) `global_spell_id` is increased (eg. to 2)

5) spell display entities are spawned (in this case 3 entities)



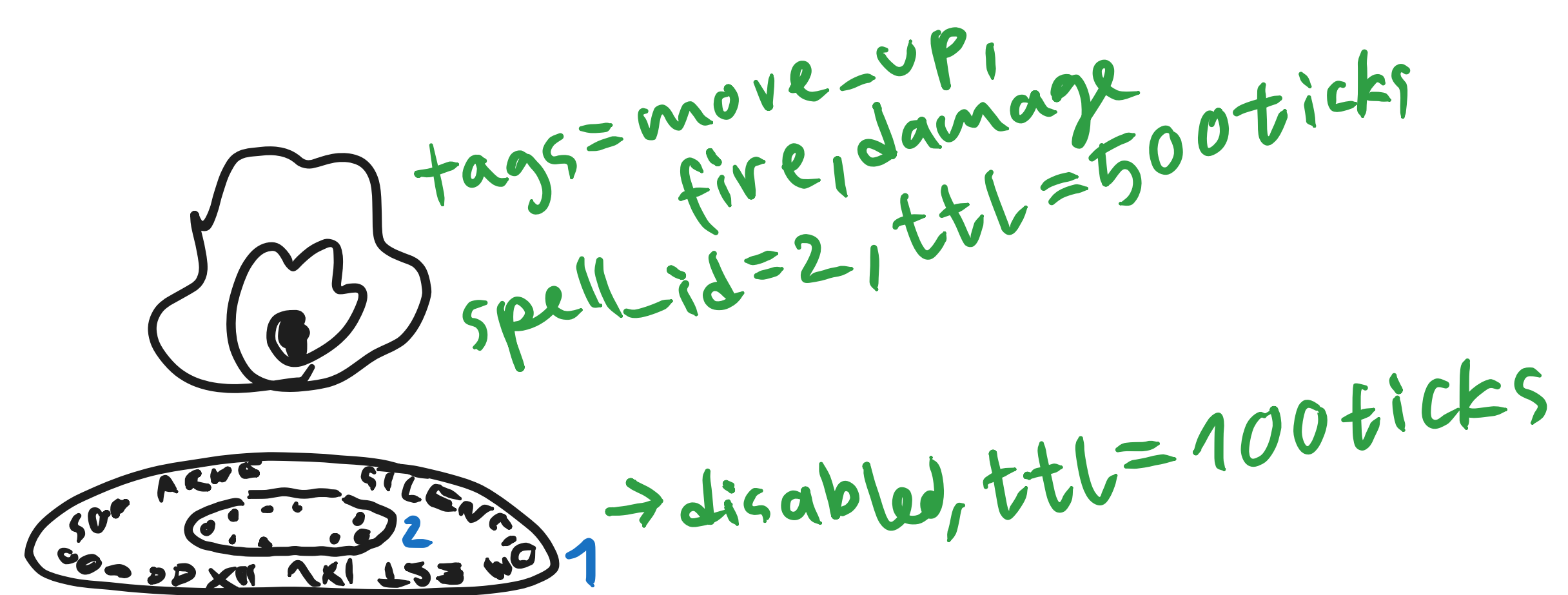
6) player triggers execution

7) the `custom_data.spell` is stored back into `data.spell`

8) `execute.mfunction` is called as the outmost entity

9) all entities with `spell_id=2` get the tag `disabled`

10) New entities are summoned (some old ones may be killed)



11) `effect.mfunction` runs every tick and applies to `move-up`, `fire`, `damage` & `ttl`

12) After 100 ticks, `spell_id=2` and `disabled=true` is killed.

13) After 500 ticks, `spell_id=2` and `disabled=false` is killed

note: specific implementations might change, like
- using the new schedule .. append instead of TTL,
- always using the data storage vs executing a phase as the outmost entity...
the most important thing is to do it so that the spell logic can be very free